

COS214 Project Report: Smart Greenhouse Management System

Group members: Josua Louw (u24569772), Daniel Darius Erasmus (u24586189), Nico Theron (u23565722), Heindrich Jansen (u24711358), Frikkie Malan (u14439141)

Abstract

The Smart Greenhouse Management System was developed as part of the COS214 Design Patterns module to demonstrate advanced object-oriented design principles through a cohesive, pattern-rich software solution. The project simulates the daily operations of a greenhouse nursery, which includes inventory management, customer ordering, staff coordination, and plant lifecycle simulation. Through the use of multiple design patterns (13 in total), the system achieves modularity, reusability, extensibility, and clear separation of concerns. To enhance realism and system performance, concurrency was incorporated to simulate simultaneous operations such as plant growth, order processing, and section updates, demonstrating how multithreading can improve responsiveness and scalability in software systems. Comprehensive testing was conducted using the Catch2 framework to ensure functional correctness and code coverage.

Research Brief

Running a plant nursery requires balancing biological processes (growth, watering, fertilization) with logistical concerns (inventory, sales, customer requests). For a software simulation, these concerns translate into entities that change state over time (plants), resources that must be tracked (stock levels, pots, decorations), and concurrent operations (care tasks, sales) that must be coordinated. This project models those realities using object-oriented design and established software patterns to produce a maintainable, extensible simulator.

Research into horticultural science shows that plants progress through identifiable stages (seed, seedling, juvenile, mature, flowering, senescence, dying, dead).¹ Each stage has different care needs (water, nutrients, pruning) and time windows for actions. Modelling discrete states simplifies control logic and enables realistic transitions.^{1,2} To this end, the State and Strategy design patterns seem most appropriate to adequately model this behaviour. Related classes of plants are then typically grouped together in nurseries to accommodate for easier identification and ease of similar care conditions that could be applied (*i.e.* sunlight, water and feeding requirements). For this, modelling a greenhouse in a Composite design pattern seems appropriate, along with incorporating an Iterator pattern to efficiently be able to

search for specific items. The Prototype pattern allows for simple plant replication as it would in a typical nursery. In commercial nurseries, plants are often sold alongside configurable accessories (pots, decorations, soil types). This motivated the use of the Decorator and Builder patterns to support composable product structures and efficient order construction. Treating base items (seeds, plants) and decor/containers as composable components mirrors real-world bundling and supports price composition where unique orders are produced.

Research into business process management highlights the importance of task distribution and event communication.³ The Command, Chain of Responsibility, and Mediator patterns reflect these principles by decoupling task execution, allowing caretakers, managers, and customers to interact seamlessly through a central hub. In a running nursery, multiple actions occur simultaneously: plants grow over time while staff fulfil orders and customers browse. Multithreading lets us simulate these parallel activities (growth timers, scheduled care, order processing) and test synchronisation/consistency under realistic loads. Careful locking or message-based coordination (Mediator/Command) prevents race conditions while keeping the model responsive. Test-driven development and automated coverage tools (e.g. gcov) were applied to ensure behavioural correctness and completeness of coverage across all of the implemented classes and class hierarchies.

Assumptions and scope limitations

Biological timings are simulated (discrete ticks) rather than matching real calendar time and are unique according to the plant types. The concurrency models used focus on demonstrating correctness and race-safety in the simulator, where full real-time scheduling and persistence are out of scope. Pots, decorations, and seed packets are also viewed as sellable items, and need not be sold with a plant. However, it is possible to “decorate” a given plant with a pot or decoration as part of the order. The simulator emphasises design-pattern demonstration and unit-test coverage over graphical UI.

Design Pattern Application – System Overview

The system models a greenhouse that stores various items such as plants, pots, decorations, and seeds. Each plant progresses through a set of states (e.g., seed → seedling → mature → flowering → dying), requiring care actions at different stages. The greenhouse supports the management of items and sections (inventory subsystem), processing of customer orders, staff coordination using task assignment and communication, concurrent plant growth

simulation, reflecting real-time changes in state and required care. These subsystems interact through structured communication and design pattern integration.

Functional Requirements

1. Maintain an organised greenhouse inventory with items grouped into sections.
2. Support customer orders involving customisable items (decorations, pots, seeds).
3. Allow staff (sales managers and caretakers) to execute and delegate tasks.
4. Simulate plant lifecycles and care routines over time.
5. Display summaries and iteratively traverse all items within sections.

Non-Functional Requirements

1. Extensibility: New item types and staff roles can be added easily.
2. Reusability: Patterns allow common logic to be reused (e.g., iteration, cloning).
3. Maintainability: Strong abstraction boundaries between components.
4. Concurrency: Real-time plant behaviour and growth simulation.

Design Patterns Implemented

1. Composite Pattern

Purpose: Represent the hierarchical structure of the greenhouse. Used to build the greenhouse with plants and subsections.

Classes: GreenHouse (Component), Section (Composite), Item (Leaf).

Note: Item is abstract, and therefore the concrete Leaf classes have been included for a complete representation of the pattern.

Benefit: Allows uniform treatment of individual items and groups of items, as well as order them in sections according to type.

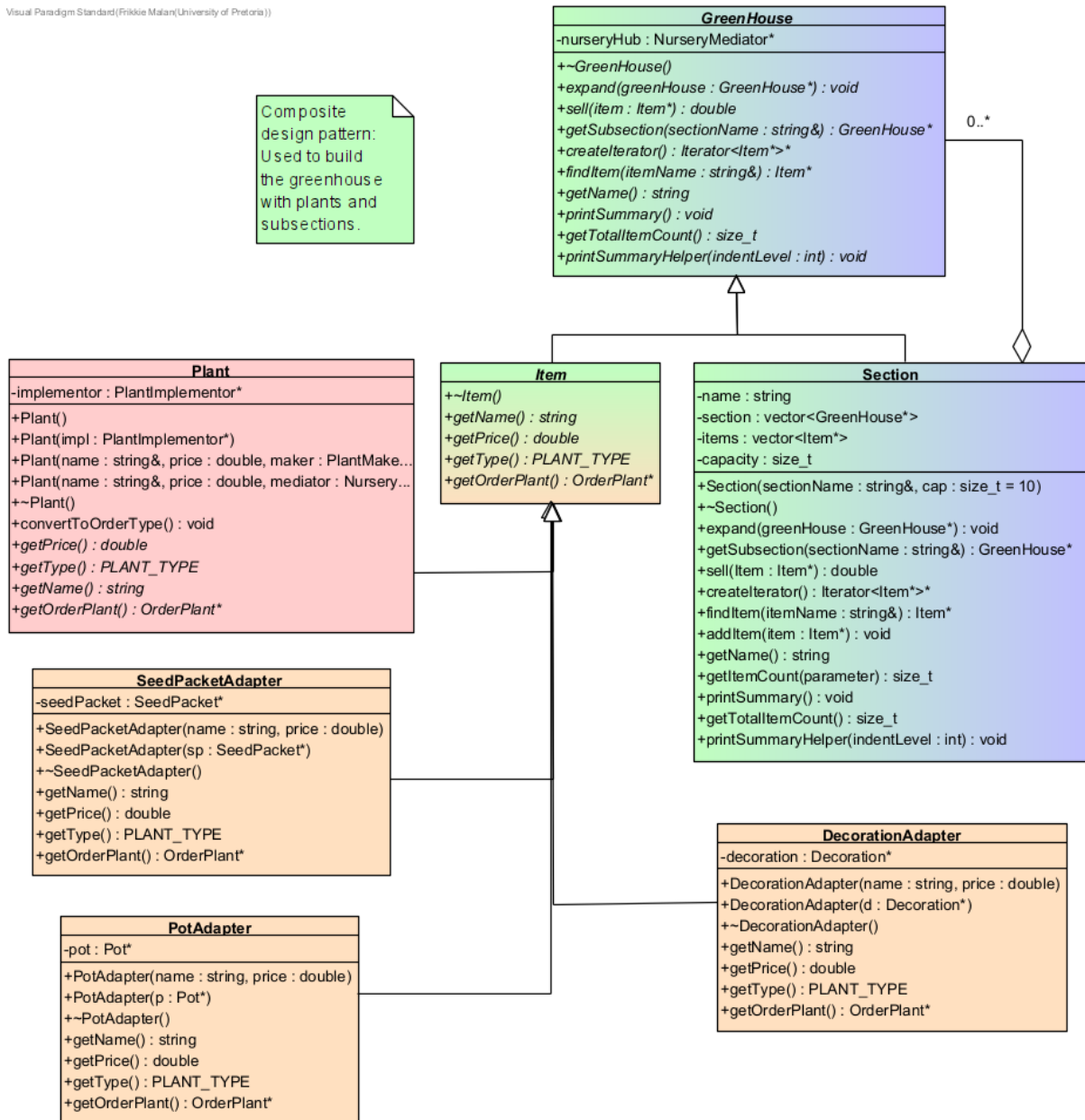


Figure 1. Composite pattern.

2. Iterator Pattern

Purpose: Traverse greenhouse contents see each plant without exposing internal structures.

Classes: GreenHouse (Aggregate), Section (ConcreteAggregate), ItemIterator (ConcreteIterator), Iterator (Iterator).

Benefit: Simplifies traversal across nested sections and subsections.

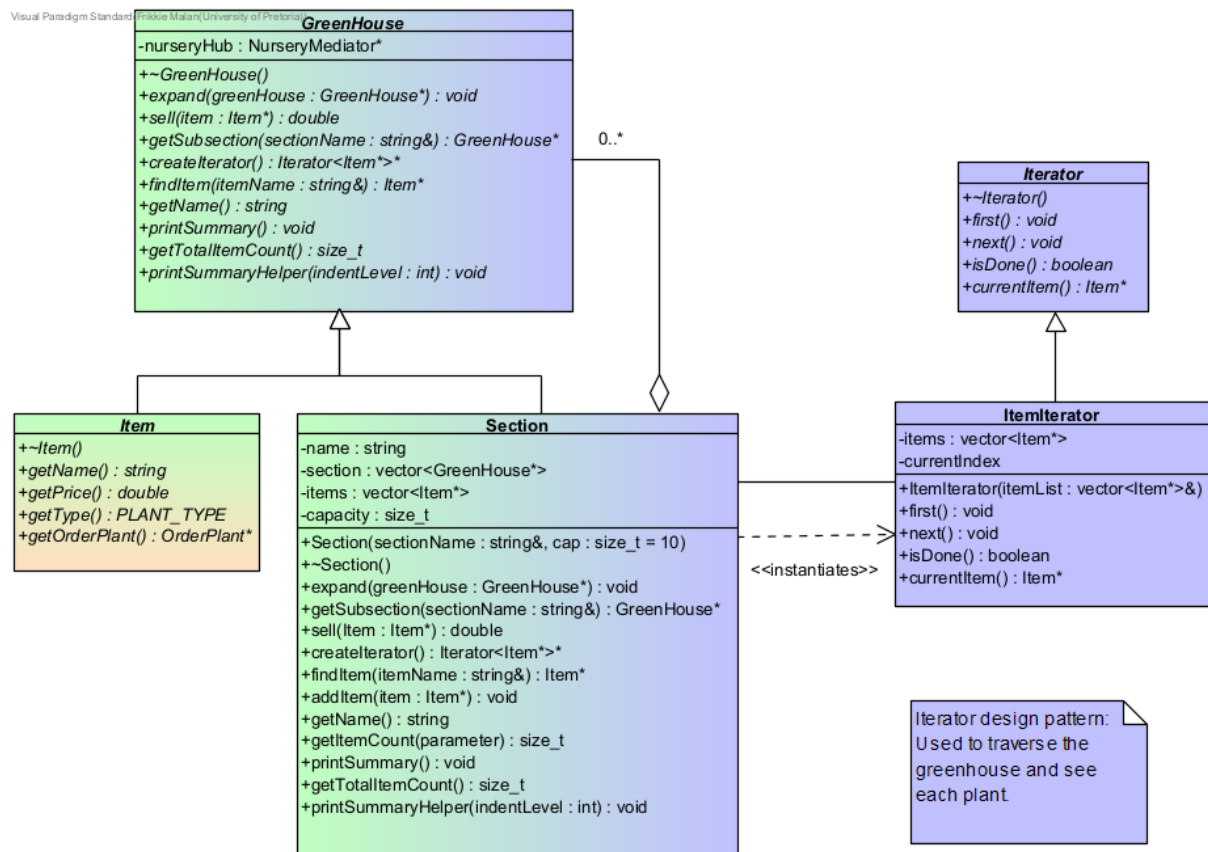


Figure 2. Iterator pattern.

3. Builder Pattern

Purpose: Construct complex orders step-by-step for customers.

Classes: Customer (Customer), OrderBuilder (Builder), AddDecoration (ConcreteBuilder), AddPot (ConcreteBuilder), AddPlant (ConcreteBuilder), AddSeed (ConcreteBuilder), Order (Product).

Benefit: Simplifies the creation of complex, customisable orders.

4. Prototype Pattern

Classes: OrderPlant (Prototype), PlantType (ConcretePrototype), Decoration

Benefit: Reduces object creation overhead for repetitive plant creation.

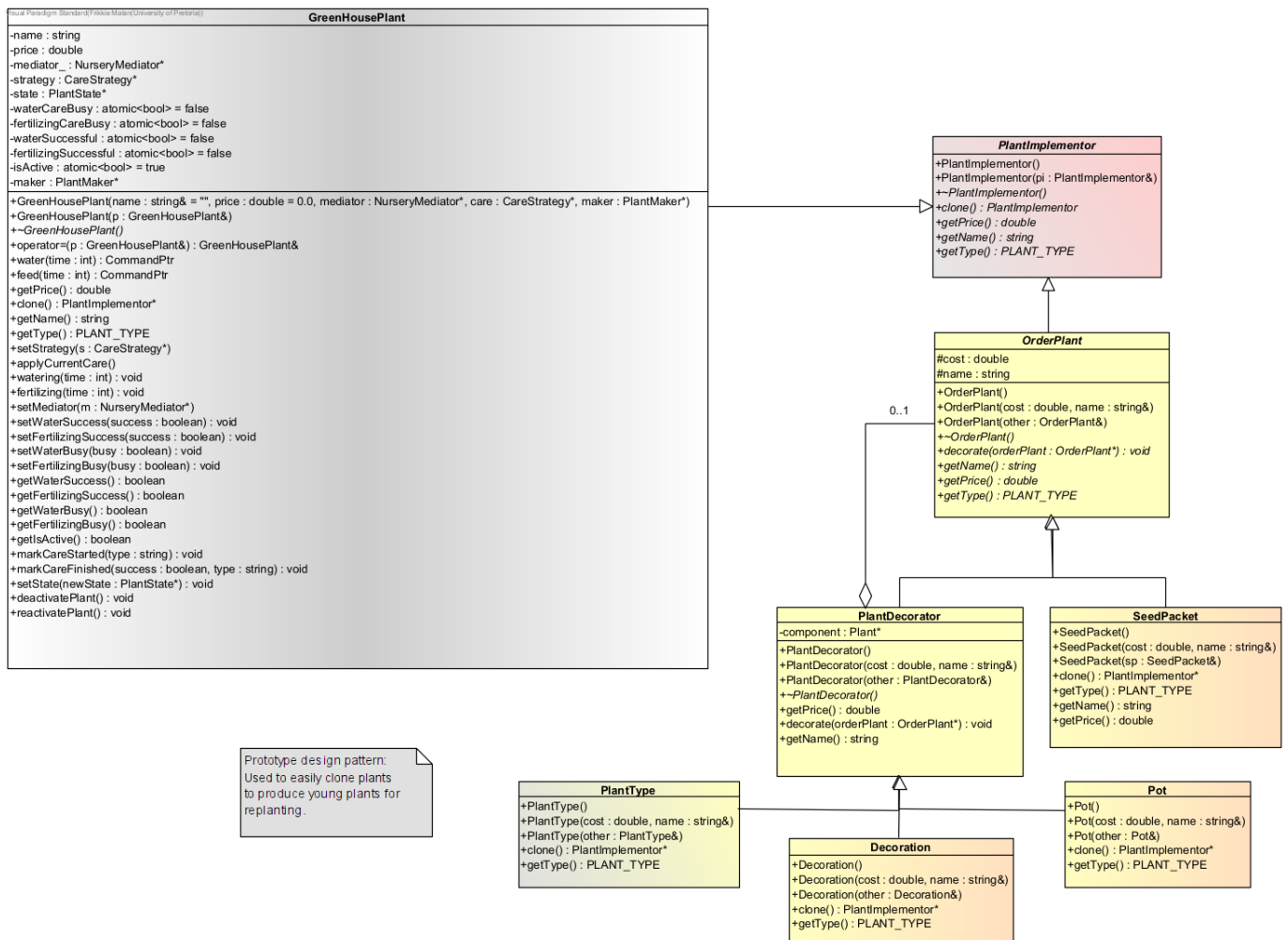


Figure 4. Prototype pattern.

5. Adapter Pattern

Purpose: Integrate diverse item types (pots, seeds, decorations) as sellable items.

Classes: Item (Target), SeedPacketAdapter (Adapter), PotAdapter (Adapter),

DecorationAdapter (Adapter), SeedPacket (Adaptee), Pot (Adaptee), Decoration (Adaptee).

Benefit: Allows reusing existing classes without altering their interfaces.

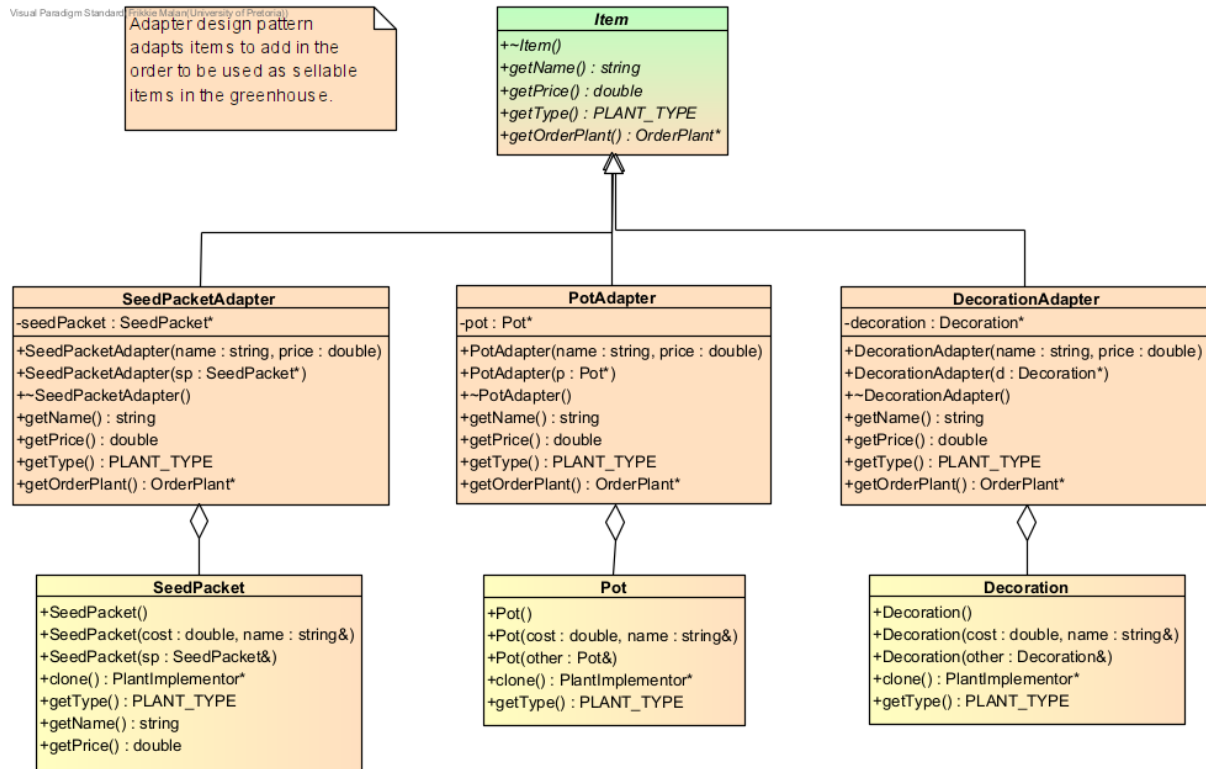


Figure 5. Adapter pattern.

6. Decorator Pattern

Purpose: Used to make unique plant types by adding optional features to plants, such as pots and decorations.

Classes: OrderPlant (Component), PlantDecorator (Decorator), PlantType (ConcreteDecorator), Decoration (ConcreteDecorator), Pot (ConcreteDecorator), SeedPacket (ConcreteComponent).

Benefit: Enables customisation of plants in customer orders.

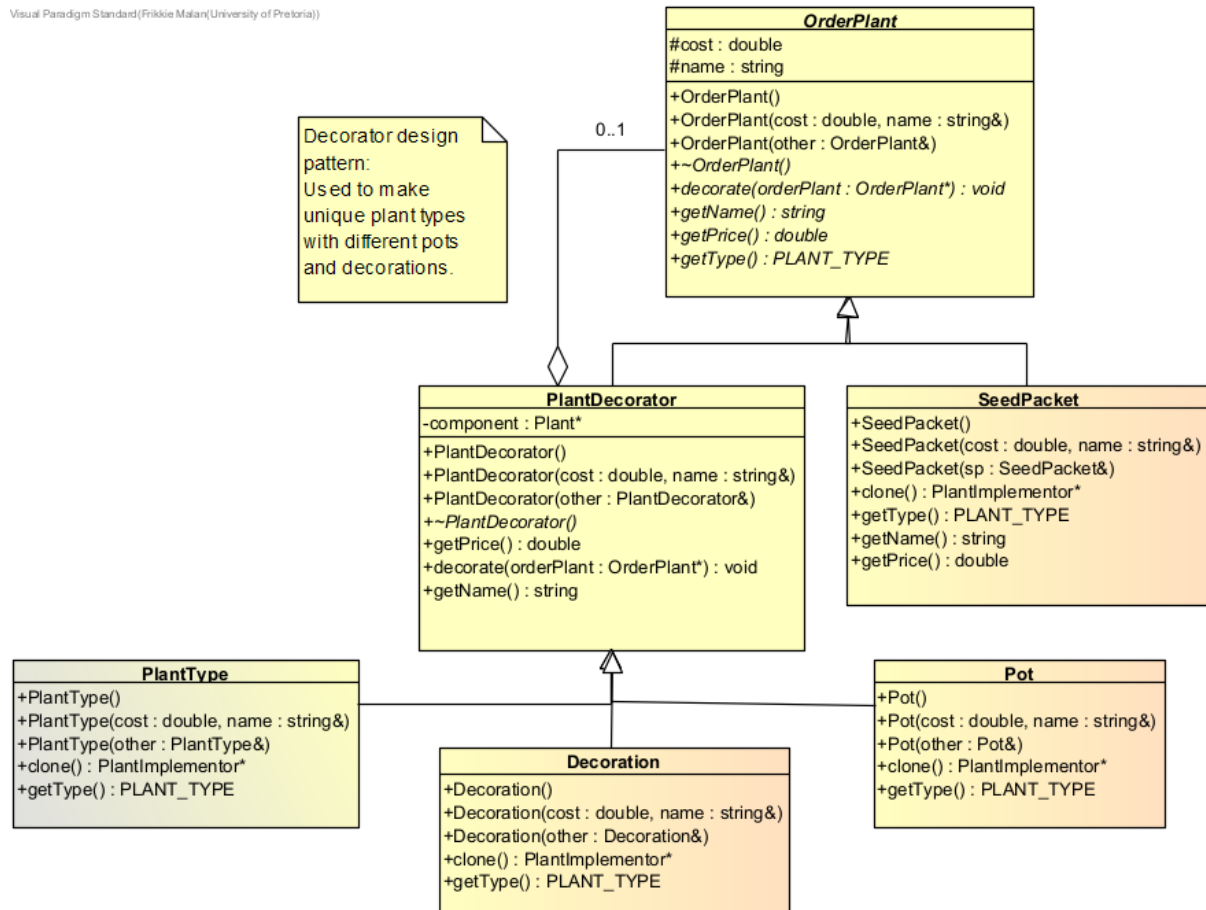


Figure 6. Decorator pattern.

7. Bridge Pattern

Purpose: Separate plant representation in the greenhouse and customer orders.

Classes: Plant (Abstraction), Order (RefinedAbstraction), PlantImplementor (Implementor), GreenHousePlant (ConcreteImplementor), OrderPlant (ConcreteImplementor).

Note: OrderPlant is abstract, and therefore the concrete classes inheriting from OrderPlant have been included for a complete representation of the pattern.

Benefit: Increases flexibility between plant abstraction and implementation.



Figure 7. Bridge pattern.

8. Strategy Pattern

Purpose: Decide dynamically what care strategy a plant needs.

Classes: GreenHousePlant (Context), CareStrategy (Strategy), FertilizerBoostStrategy (ConcreteStrategy), RegularCareStrategy (ConcreteStrategy), WaterLimitingStrategy (ConcreteStrategy).

Benefit: Promotes flexible plant maintenance logic.

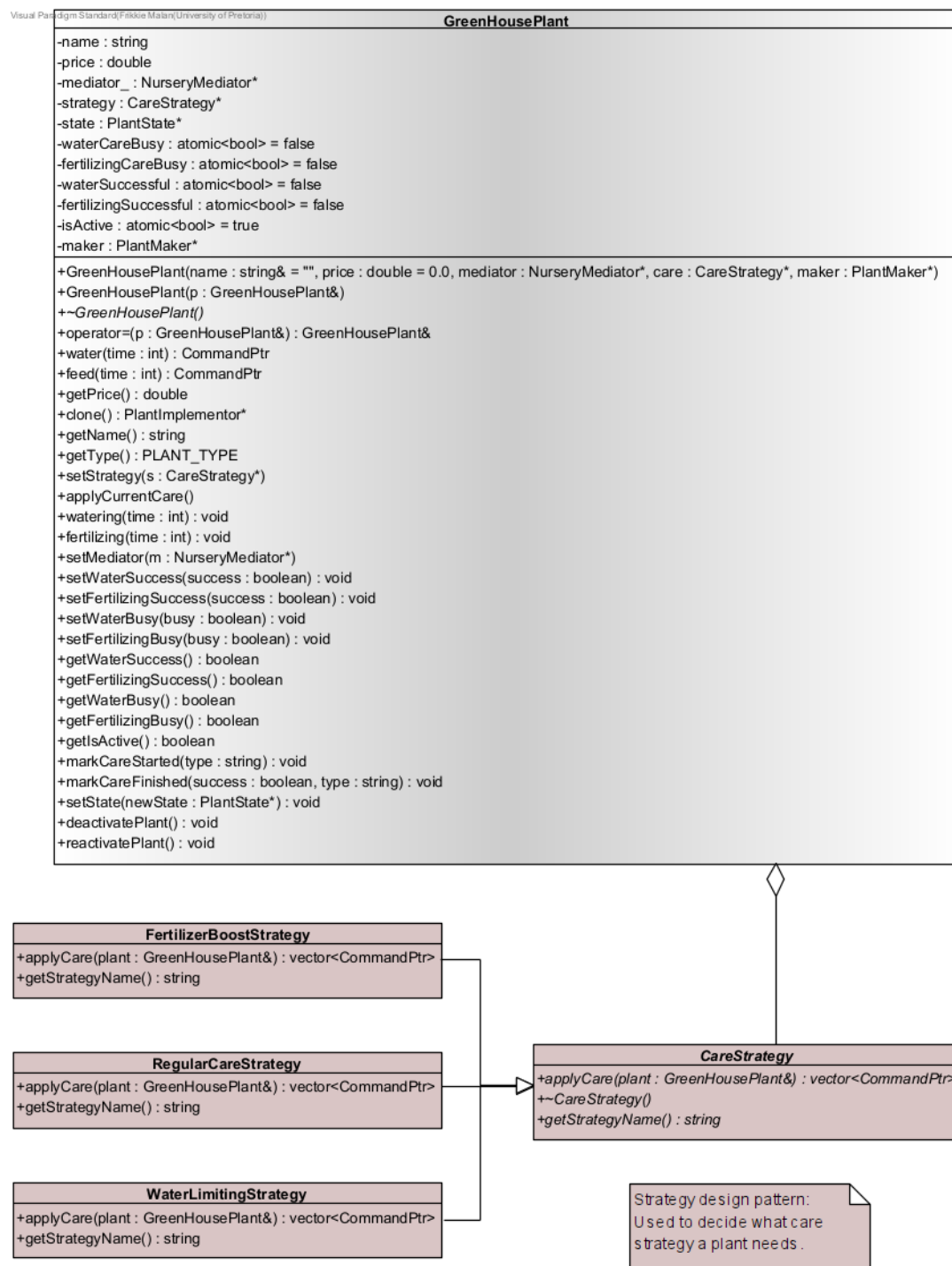


Figure 8. Strategy pattern.

Purpose: Represent and execute plant care or sales commands by staff members.

Classes: Staff (Invoker *via* SalesManager and PlantCaretaker), Command (Command), WaterPlant (ConcreteCommand), FertilizePlant (ConcreteCommand), SellCommand (ConcreteCommand), Customer (Receiver), GreenHousePlant (Receiver).

Benefit: Decouples request invokers from executors.



Purpose: Delegate tasks among staff members.

Classes: Staff (Handler), SalesManager (ConcreteHandler), PlantCaretaker (ConcreteHandler).

Benefit: Enables flexible staff task distribution.

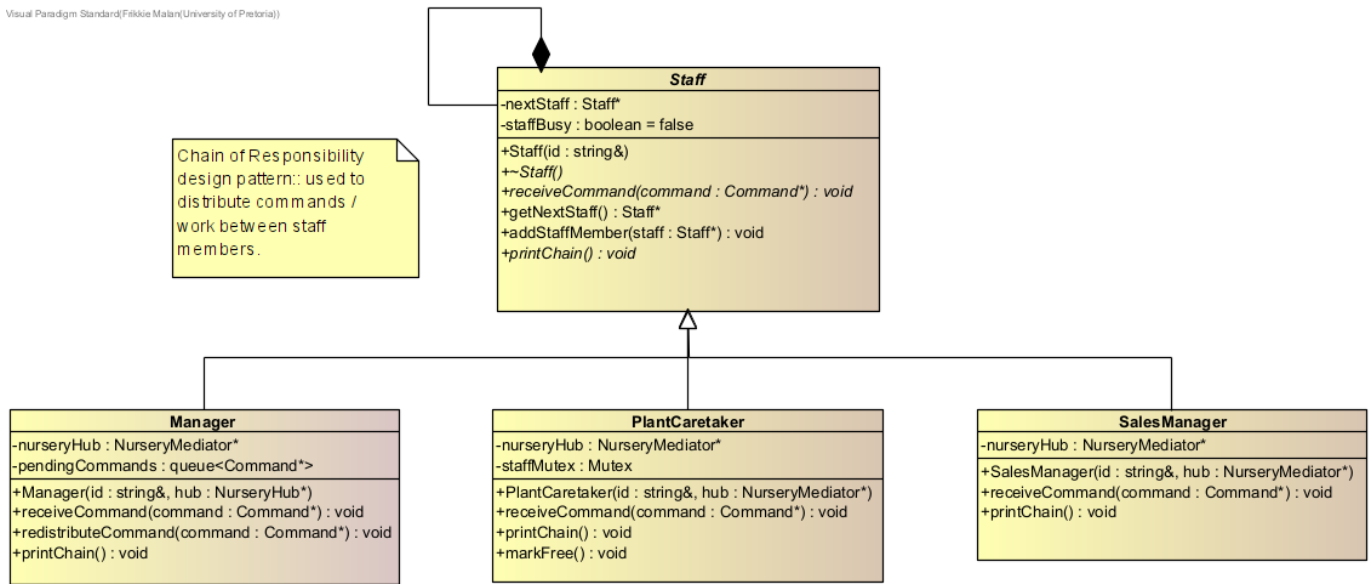


Figure 10. Chain of Responsibility pattern.

11. Mediator Pattern

Purpose: Coordinate communication between staff, customers, and plants.

Classes: Staff (Colleague), SalesManager (ConcreteColleague), PlantCaretaker (ConcreteColleague), NurseryMediator (Mediator), NurseryHub (ConcreteMediator).

Benefit: Centralises communication, reducing coupling.

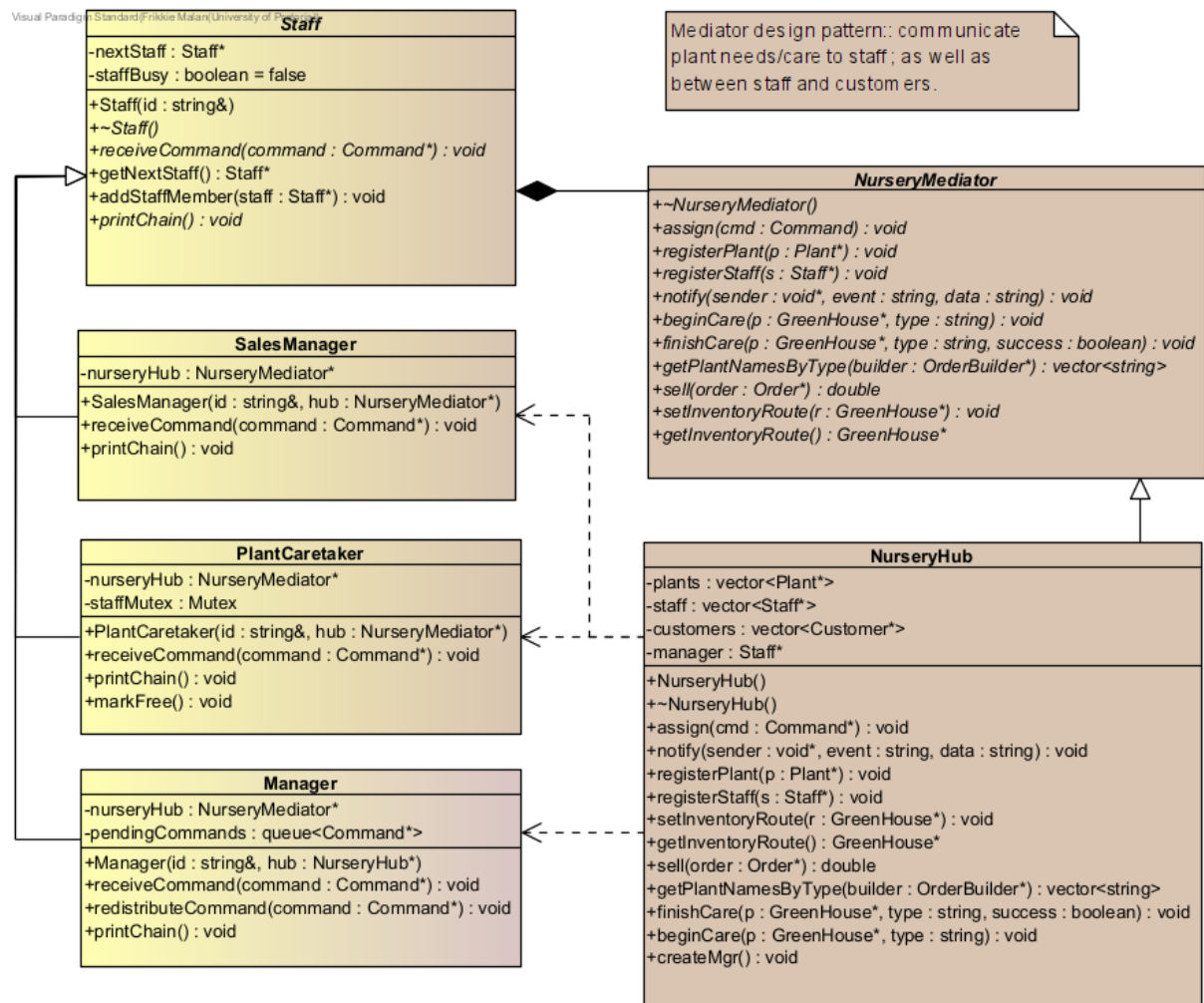


Figure 11. Mediator pattern.

12. State Pattern

Purpose: Manage plant lifecycle and transitions between states.

Classes: GreenHousePlant (Context), PlantState (State), the following classes are ConcreteState: SeedState, SeedlingState, JuvenileState, MatureState, FloweringState, SenescenceState, DyingState, DeadState.

Benefit: Models plant growth and transitions clearly.

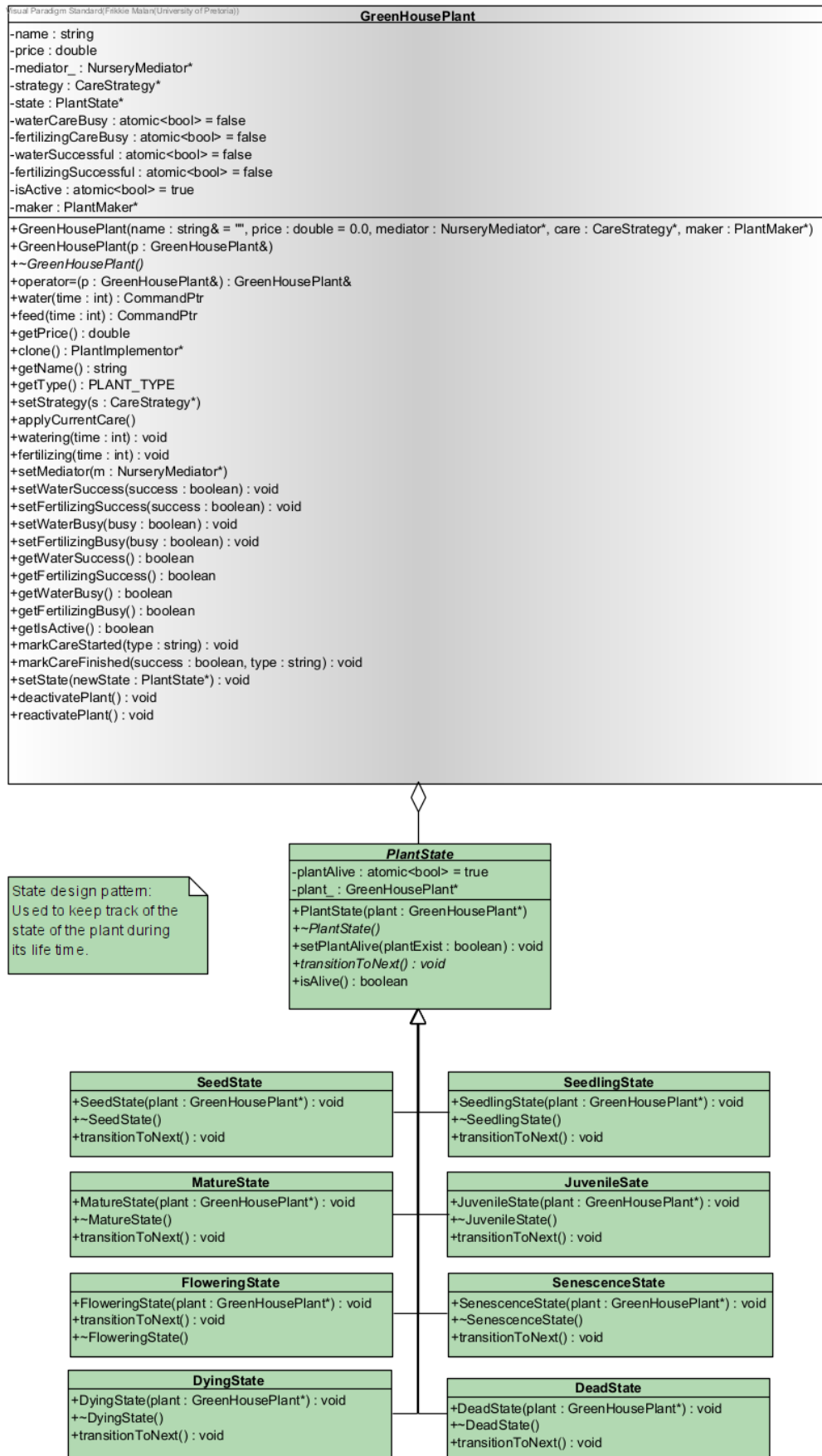


Figure 12. State pattern.

12. Façade Pattern.

Purpose: Provides the unified interface (GUI) to a set of interfaces in a subsystem.

Classes: GreenHousePlant (Façade), Subsystem classes: Remainder of classes.

Benefit: Simplifies the overall use of the system as a whole by providing a simplistic interface to alleviate the user of manually calling/invoking functions in a text-based manner.

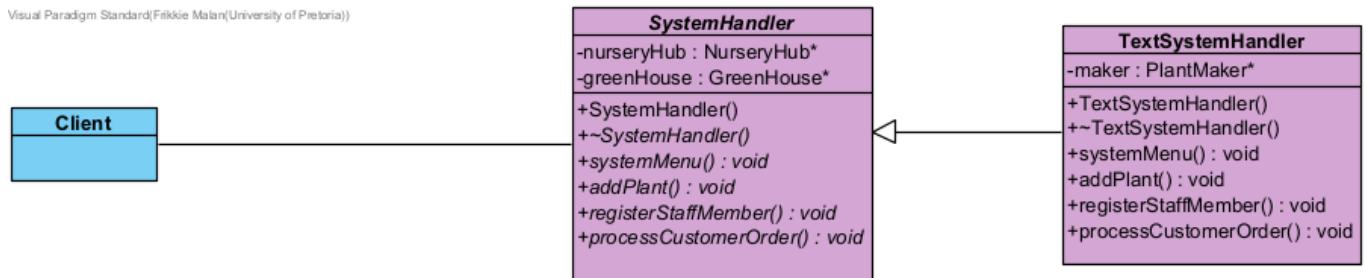


Figure 13. Façade pattern.

Testing and Evaluation

Testing was performed using doctest and Valgrind for memory safety. All tests passed successfully, with no detected memory leaks. Coverage analysis (via gcov) confirmed execution of all classes.

Conclusion

This project successfully integrates 13 classical design patterns into a single, cohesive software system. Through these patterns, the group as a whole demonstrated a deep understanding of object-oriented design, proficiency in modular architecture, and the ability to integrate concurrency and stateful behaviour. The final system effectively simulates a functioning greenhouse environment, which includes inventory management, staff coordination, and dynamic plant growth, and represents a strong showcase of design pattern application at scale.

Literature references

1. Learn The Six Plant Growth Stages, web article, accessed 02/11/2025.
<https://www.saferbrand.com/articles/plant-growth->

stages?srsId=AfmBOooVJn1W6aZBP2kWuA3E-96H9CskT-tOrK6KTqE6-Xmb-8acTeCx.

2. Flowering plant life cycles, web article, accessed 02/11/2025. <https://www.sciencelearn.org.nz/resources/82-flowering-plant-life-cycles>.
3. B. Ozkan, M. Koops, O. Türetken, H. A. Reijers, The Influence of Business Process Management System Implementation on an Organization's Process Orientation: A Case Study of a Financial Service Provider, *Information Systems Management*, 2024, 41, 377-398. <https://doi.org/10.1080/10580530.2023.2286980>.